Assignment 2: Technical Design

1. Project Identification

Team Name	Nazal
Team Members	Naman Anshumaan and Mohammad Fazlur Rahman
Roll Numbers	1020241465 and 1020242018
Assigned Project	Interactive Multi-Track Audio Editor and Mixer
Repository Link	`https://github.com/Ashoka-CS-3810-Design-Practices-in-CS/project-1-nazal.git`

Project Title: Interactive Multi-Track Audio Editor and Mixer

Problem Statement

Editing and arranging audio typically requires specialized software that can be complex for users who have no experience with highly used professional digital audio editors. The project aims to develop an easy-to-use multi-track audio editor that allows users to import audio files, arrange them on a timeline, edit individual clips, mix multiple tracks, apply basic effects, and export the final composition. Users will be able to work on a project as a collection of multiple audio clips positioned on a timeline. They can directly manipulate these clips through an easily understandable UI while simultaneously controlling audio properties such as volume, mute, and solo.

Motivation

The project is meant to combine audio editing with interactive software design. The project enables the ability to visually comprehend audio editing, instead of treating audio processing as a sequence of command-line operations. The project also provides an opportunity to apply concepts from data structures, algorithms, concurrency, data representation, and modular software architecture to a practical application.

Intended users

Our intended users are the one who want to perform basic audio editing and mixing without requiring a high level audio workstation, which usually takes some time to understand. Examples of the users could be students working on projects, beginners wanting to learn basic audio editing, users creating simple music compositions, users editing voice recordings, podcasts, or other multi-track audio and users who need to combine and arrange multiple audio recordings.

Main goal

The main goal is to develop an intuitive, responsive, and modular application that allows users to create a complete multi-track audio composition by importing, arranging, editing, mixing, and exporting audio. The application should integrate the timeline, waveform visualization, audio playback, editing operations, effects, mixing controls, and export system into a single interface.

1 Algorithms

Algorithm 1: Active clip detection

Problem solved

During playback, the system needs to determine which clips should currently be playing at a particular timeline position.

Input

- Current playback time t
- Collection of tracks
- Clips belonging to each track
- Each clip's start time and duration

Each clip can be represented by $[\texttt{startTime}, \texttt{endTime})$ where $\texttt{endTime} = \texttt{startTime} + \texttt{duration}$.

Output

The collection of clips that are active at time t.

High-level steps

```
findActiveClips(t):
    activeClips = empty list
    for each track:
        for each clip in track:
            start = clip.startTime
            end = clip.startTime + clip.duration
            if start <= t and t < end:
                add clip to activeClips
    return activeClips
```

Correctness intuition

A clip should play exactly when the current timeline position lies between its start and end positions. Basically, checking start $\leq t <$ end correctly determines whether the clip is active.

Complexity

If there are N total clips:

- Best case: $O(N)$
- Average case: $O(N)$
- Worst case: $O(N)$
- Space: $O(K)$, where K is the number of active clips.

Why this algorithm?

A linear scan is quite basic and is appropriate for the expected scale of our project. It gets the job done. More advanced interval trees could improve repeated searches, but would add unnecessary implementation complexity.

Algorithm 2: Clip splitting

Problem solved

Splitting allows a user to divide one audio clip into two separate clips at a selected timeline position.

Input

- Original clip
- Split position t

Output

Two clips representing the two portions of the original clip on the timeline itself.

High-level steps

Let's say that a clip has start = 10s, end = 20s and the user splits at 15 seconds.

```
splitClip(clip, t):
    verify start < t < end
    create leftClip
        leftClip.start = clip.start
        leftClip.end = t
    create rightClip
        rightClip.start = t
        rightClip.end = clip.end
    calculate corresponding source-audio offsets
    replace original clip with leftClip and rightClip
```

Correctness intuition

The two new clips cover (start, t) and (t, end). Together they cover exactly the original clip's timeline region without changing the underlying source audio.

Complexity

Assuming the clip object and project collection support direct replacement:

- Best: $O(1)$
- Average: $O(1)$
- Worst: $O(N)$
- Space: $O(1)$

Why this algorithm?

The project is implementing a non-destructive editing model. Splitting the clip's representation instead of modifying the original audio file makes the operation simple and preserves the source audio.

Algorithm 3: Waveform generation

Problem solved

The system needs to convert raw audio tracks into a comprehensible visual representation that can be displayed on the timeline.

Input

- Audio sample data
- Number of waveform display regions/pixels

Output

A continuous sequence of amplitude ranges used to draw the waveform.

High-level steps

```
generateWaveform(samples, numberOfRegions):
    divide samples into approximately equal regions
    for each region:
        minimum = minimum sample amplitude
        maximum = maximum sample amplitude
        store (minimum, maximum)
    return waveform data
```



Figure 1: Waveform generation[1]

[1]This figure was generated using AI.

Correctness intuition

The minimum and maximum values preserve the amplitude range of each region of the audio signal. Drawing these ranges in sequence provides a visual approximation of the original waveform.

Complexity

If there are S audio samples:

- Best: $O(S)$
- Average: $O(S)$
- Worst: $O(S)$
- Space: $O(P)$, where P is the number of stored waveform regions.

Why this algorithm?

Processing every sample once is simple and provides the information required for visualizing the waveform. Our design already specifies grouping samples into timeline regions and representing them using minimum and maximum amplitudes.

Algorithm 4: Multi-track audio mixing

Problem solved

Multiple tracks need to be combined into a single audio output during playback and export.

Input

- Active clips
- Audio samples from each clip
- Track volume
- Mute/solo state
- Fade settings

Output

A single mixed audio clip.

High-level steps

```
mixTracks(activeTracks):
    create empty output buffer
    determine whether any tracks are soloed
    for each track:
        if track is muted:
            continue
        if another track is soloed and this track is not soloed:
            continue
        for each active audio sample:
            sample = sample * track.volume
            apply fade gain if required
            add sample to output buffer
    return output buffer
```

Correctness intuition

Each track contributes its audio to the final output only if it is allowed to play. Its samples are first scaled by the track's volume and any applicable fade gain. The resulting samples are then summed with the corresponding samples from other active tracks. Hence, the output represents the combined contribution of all active tracks.

Complexity

If there are T active tracks and each contributes S samples, the straightforward implementation takes $O(TS)$ time. Space is $O(S)$ for the output audio buffer, ignoring the memory already required for the source audio.

Why this algorithm?

Direct sample mixing is simple and matches the requirements of the application. It also allows volume, mute, solo, and fade operations to be applied at the correct point before the final mix.

2 Data Structures

DS1: Project

Stored information

This one basically represents the complete process and outcome. It contains the project name, collection of tracks, project duration, audio-file references, project-level and metadata where required.

Why is it suitable?

The entire application revolves around a project containing multiple tracks. It provides a single top-level object through which the UI, playback system, saving system, and export system can access the project state.

Important operations

- create project
- add/remove track
- access tracks
- save project
- load project

Complexity

If tracks are stored in a standard vector, then:

- Access by index: $O(1)$
- Add at end: $O(1)$
- Remove/search: $O(T)$

where T is the number of tracks.

Interaction

The Project contains tracks, while the project manager controls creation, saving, and loading.

DS2: Track

Stored information

Each track contains the track name, collection of clips, volume, mute state and solo state.

Why is it suitable?

The requirements state that multiple independent tracks must exist and that each track needs its own clips and audio controls.

Important operations

- add clip
- remove clip
- find clips
- modify volume
- toggle mute
- toggle solo

Complexity

Using a standard vector of clips:

- Access: $O(1)$
- Add at end: $O(1)$
- Search: $O(N)$
- Remove: $O(N)$

where N is the number of clips in the track.

Interaction

Tracks belong to a project and contain clip objects. The audio engine reads track settings when generating playback or export audio.

DS3: Clip

Stored information

A clip represents a region of an audio file on the timeline. It stores source audio reference, timeline start position, duration, source start offset, fade-in duration and fade-out duration.

Why is it suitable?

This is the most important data structure for the editor as the editing can be represented by changing metadata rather than modifying the original audio file. For example, trimming changes the clip's boundaries at start and end rather than deleting samples from the actual source audio.

Important operations

- move
- trim
- split
- remove
- change fade
- determine whether active at time t

Complexity

The metadata modifications will have a complexity of $O(1)$ and finding a clip among N clips would be $O(N)$.

Interaction

Clips belong to tracks and are used by the timeline, waveform renderer, audio engine, and export system.

DS4: Audio Buffer / Waveform Data

Stored information

The application needs a representation of the actual audio samples being processed. For waveform display, it also stores derived amplitude information such as (minAmplitude, maxAmplitude) for each display region.

Why is it suitable?

Audio is naturally represented as a sequence of samples. A buffer provides direct access to these samples for playback, waveform generation, mixing, and effects. JUCE provides audio buffer abstractions that can be used instead of implementing raw sample storage from scratch.

Important operations

- read samples
- write samples
- process samples
- calculate amplitude ranges
- copy audio regions

Complexity

Accessing a sample is $O(1)$, processing S samples is $O(S)$ and generating a waveform from S samples is $O(S)$.

Interaction

The Audio Buffer stores the actual audio samples and provides the data needed for playback, waveform generation, effects, and export. It acts as the common audio-data layer connecting these components.

3 Software Architecture and Design

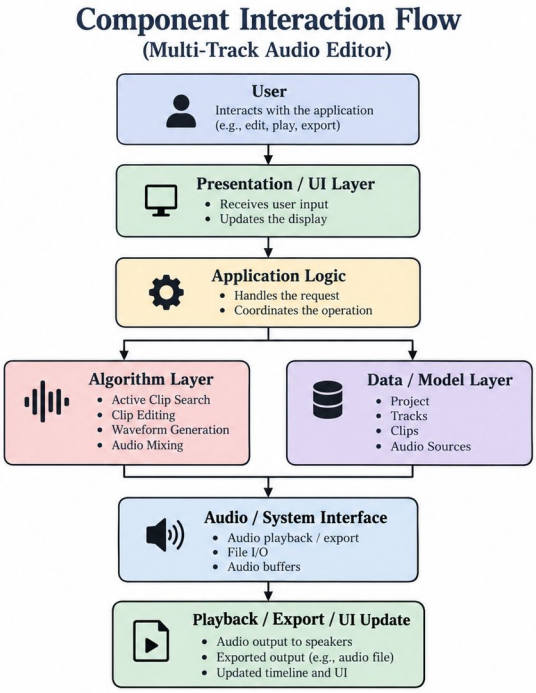


Figure 2: Architecture diagram[2]

Presentation/UI Layer

Will be responsible for

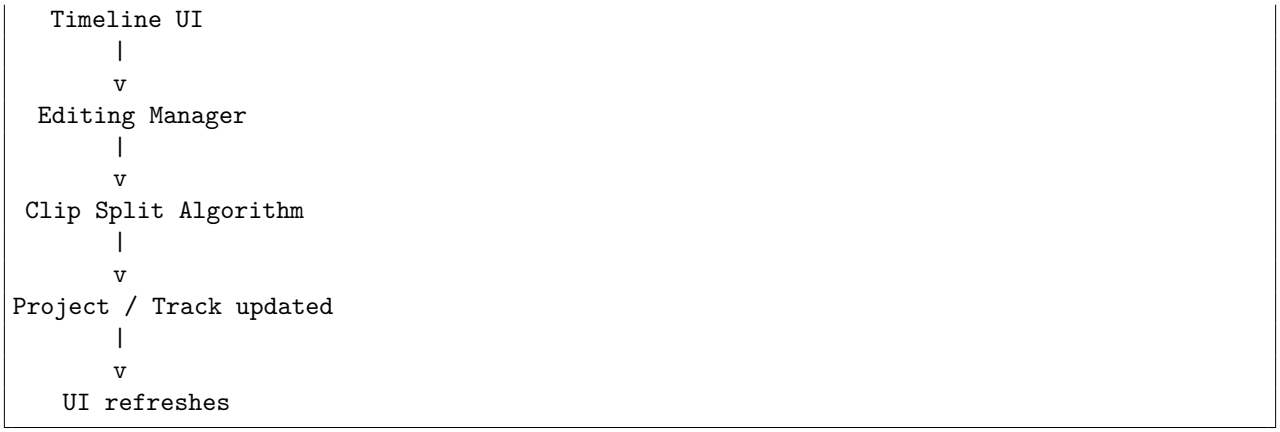
- timeline
- waveform display
- track controls
- play/pause/stop controls
- volume controls
- project controls
- export interface
- user feedback

Core application logic

This layer connects user actions with the underlying model. For example:

```
User clicks "Split"
       |
       v
```

[2]This figure was generated using AI.



Data/model layer

This layer will contain project, track, clip and audio data.

Algorithm layer

Contains the major computational procedures such as:

- active clip detection
- clip editing
- waveform generation
- mixing

Audio/system interface

JUCE will handle the interaction with:

- audio devices
- audio files
- audio buffers
- file system
- audio output
- supported audio formats

The figure below shows all the layers in interaction.

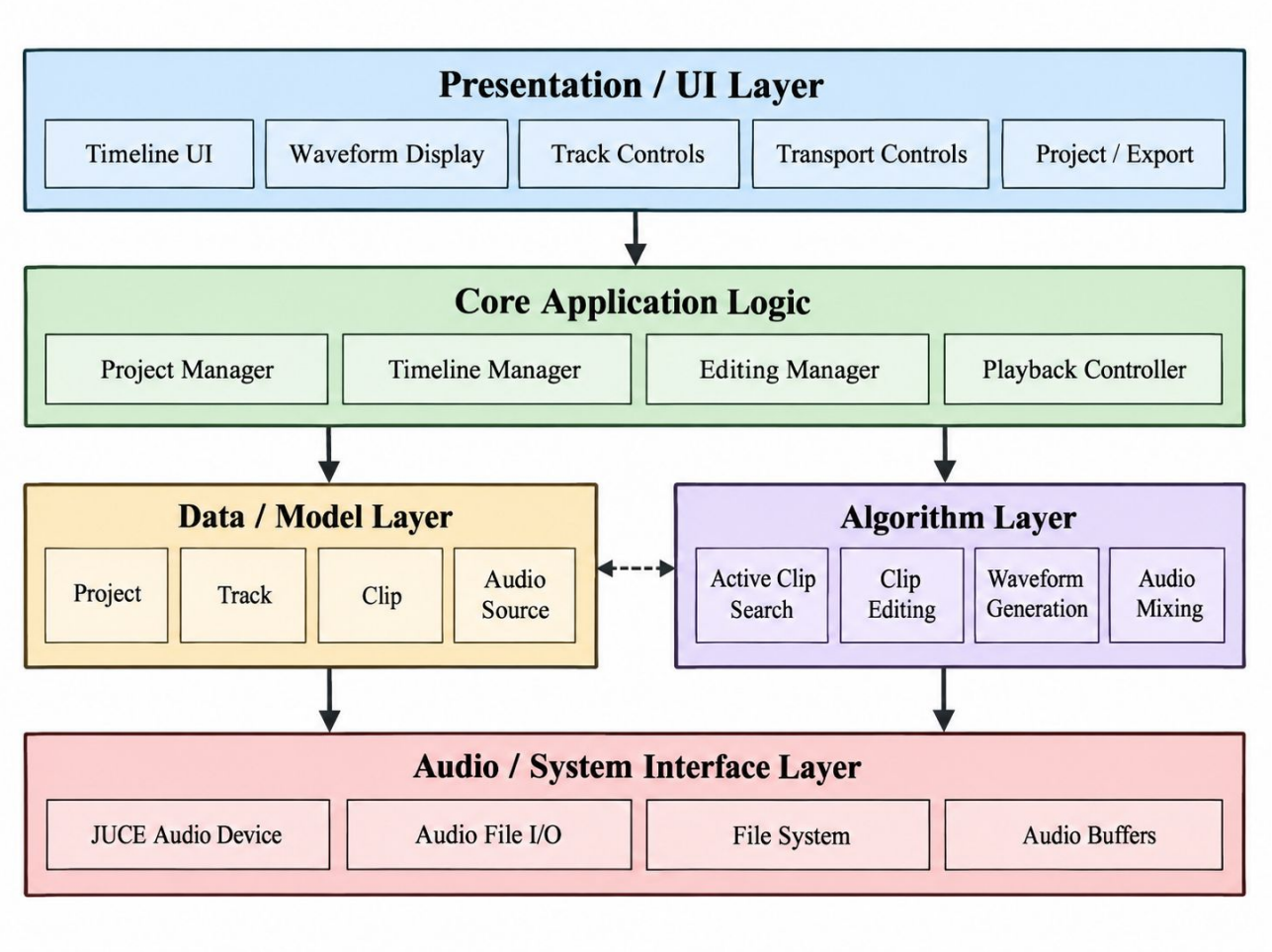


Figure 3: Layered architecture of the audio editor[3]

4 Technology Stack

For our implementation, we plan to use the following technologies. These are design choices made by our team based on the needs of the audio editor, especially real-time playback, a responsive desktop interface, and modular development.

Component	Technology	Reason for Selection
Application/UI	C++ + JUCE GUI	We will use JUCE's GUI components to build the main desktop interface, including the timeline, track controls, waveform display, and playback controls.
Core/System Components	C++	C++ gives us direct control over memory, timing, and performance, which is useful for audio playback, mixing, and project-level operations.

[3]This figure was generated using AI.

Component	Technology	Reason for Selection
Libraries/Frameworks	JUCE	JUCE provides audio device access, audio buffers, file handling, playback support, audio-format handling, and GUI components. This reduces the amount of low-level audio functionality we need to build ourselves.
Data Storage	Project file + referenced audio files	The project file will store information such as tracks, clip positions, track settings, and effects. The original audio files will be referenced rather than duplicated inside the project.
Testing	C++ unit testing + integration/system testing	Unit tests will be used for individual operations and algorithms, while integration and system testing will check whether the timeline, playback engine, UI, saving, and export work correctly together.
Version Control	Git/GitHub	We will use Git and the provided GitHub repository to maintain version history, manage changes, and collaborate during development.
Build/Package Tools	CMake + JUCE	CMake will be used to configure and build the C++ project, while JUCE will be integrated as the main application and audio framework.